

Numerical methods are used to solve ODEs when typical exact analytical solutions are difficult (tedious) or impossible to obtain using conventional methods. Instead of producing exact expressions for solutions, these methods approximate the solution by employing discrete value steps to simulate the solution given an initial condition.

Euler's method uses the slope at the current point to advance the simulation forward, but this method is not the best as the error grows the more steps that are taken. This is because we end up truncating part of the Taylor expansion.

Runge-Kutta methods improve upon Euler's method by incorporating additional slope evaluations at each step, which allows for less error due to truncation.

The primary reason why the most precise numerical methods are not employed all the time is because there are typically always trade-offs between accuracy and computational costs. For example, while higher-order methods are far more effective at reducing error than lower-order methods, sometimes lower-order methods are used as it is cheaper.

Stability is another critical issue when dealing with ODEs. With "stiff" equations, more explicit methods like Euler's method can become unstable and fail to approximate the solution well unless the step size is very small, even while the true solution is well behaved. It is for this very reason, methods like the Backward Euler's method was invented as they are able to maintain stability for much larger step sizes.

Output of Backwards Euler

```
>> run_comp(4)
t h = 0.05 h = 0.03 h = 0.01 h = 0.001 Exact
0.0 1.000000 1.000000 1.000000 1.000000 1.000000
0.1 1.692969 1.647438 1.623664 1.610463 1.609042
0.2 2.761670 2.621131 2.549137 2.509573 2.505330
0.3 4.417453 4.092089 3.928572 3.839638 3.830139
0.4 6.990552 6.320957 5.990830 5.813128 5.794226
0.5 10.996956 9.705000 9.080147 8.747267 8.712004
1.0 103.061706 80.402761 70.452395 65.419964 64.897803
1.5 959.442361 661.007307 542.124325 485.058246 479.259192
2.0 8934.069566 5435.729435 4172.722815 3597.447780 3540.200110
```

Output of 2nd Order Runge-Kutta Method

```
t h = 0.05 h = 0.03 h = 0.01 h = 0.001 Exact
0.0 1.000000 1.000000 1.000000 1.000000 1.000000
0.1 1.604975 1.607946 1.608858 1.609040 1.609042
```

```

0.2 2.493210 2.502062 2.504783 2.505324 2.505330
0.3 3.803048 3.822828 3.828915 3.830126 3.830139
0.4 5.740402 5.779689 5.791791 5.794201 5.794226
0.5 8.611750 8.684904 8.707464 8.711957 8.712004
1.0 63.424698 64.497931 64.830722 64.897114 64.897803
1.5 463.024999 474.834018 478.515883 479.251550 479.259192
2.0 3381.169838 3496.670221 3532.878861 3540.124819 3540.200110

```

Modified Code

run_comp.m

```

function run_comp(case_num)
%
if nargin < 1, case_num = 1; end
switch case_num
    case 1
        method = 'Euler';
    case 2
        method = 'Trapezoid';
    case 3
        method = 'RK4';
    case 4
        method = "BackwardEuler"; %Backwards Euler
    case 5
        method = "RK2" % 2nd Order Runge-Kutta
    otherwise
end
hs = [0.05, 0.025, 0.01, 0.001];
ts = [0, 0.1, 0.2, 0.3, 0.4, 0.5, 1.0, 1.5, 2.0];
t0 = 0;
y0 = 1;
tf = 2.0;
rhs = @(t, y) 1 - t + 4 * y;
y_true = @(t) 1/4 * t - 3/16 + 19/16 * exp(4 * t);
youts = cell(size(hs));
for k = 1 : length(hs)
    [tout, yout] = ode_inte_demo(rhs, t0, y0, tf, hs(k), method);
    youts{k} = match_output(tout, yout, ts);
end
fprintf('t      h = %0.2f      h = %0.2f      h = %0.2f      h = %0.3f\n', hs(1), hs(2), ...
        hs(3), hs(4))

```

```

for k = 1 : length(ts)
    fprintf('%0.1f    %1.6f    %1.6f    %1.6f    %1.6f    %1.6f\n',
ts(k), youts{1}(k), ...
    youts{2}(k), youts{3}(k), youts{4}(k), y_true(ts(k)))
end

ode_inte_demo.m
function [ts, ys] = ode_inte_demo(rhs, t0, y0, tf, h, method)
%
%
ts = t0 : h : tf;
ys = zeros(size(ts));
ys(1) = y0;
nf = length(ts);
for n = 2 : nf
    switch method
        case 'Euler'
            ys(n) = ys(n - 1) + h * rhs(ts(n - 1), ys(n - 1));
        case 'Trapezoid'
            rhs_nm1 = rhs(ts(n - 1), ys(n - 1));
            ys(n) = ys(n - 1) + h/2 * (rhs_nm1 + rhs(ts(n), ys(n - 1) +
h * rhs_nm1));
        case 'RK4'
            k1 = rhs(ts(n - 1), ys(n - 1));
            k2 = rhs(ts(n - 1) + h/2, ys(n - 1) + h/2 * k1);
            k3 = rhs(ts(n - 1) + h/2, ys(n - 1) + h/2 * k2);
            k4 = rhs(ts(n - 1) + h, ys(n - 1) + h * k3);
            ys(n) = ys(n - 1) + h/6 * (k1 + 2 * k2 + 2 * k3 + k4);
        case 'BackwardEuler'
            ys(n) = (ys(n-1) + h * (1 -ts(n))) / (1 - 4 * h);
        case 'RK2' %Improved Euler Method
            k1 = rhs(ts(n-1), ys(n - 1));
            k2 = rhs(ts(n-1) + h/2, ys(n-1) + h/2 * k1);
            ys(n) = ys(n - 1) + h * k2;
        otherwise
    end
end
End

```

Comments

Both methods produce relatively accurate results at low step sizes.

The 2nd Order Runge-Kutta method produced values much closer to the exact solution than the Backward Euler method. This phenomenon is even more apparent as the step sizes increased. This is to be expected due to the Runge-Kutta's expected higher accuracy due to its reduced truncation error. While the Backward Euler remained stable for all test step sizes, its prediction showed larger errors which increased directly proportional to the step size.